A LOW LEVEL DESIGN

ON

MAS: A Multi-Agent Software Engineering Team for Automated Code Generation

Submitted in the partial fulfillment of requirements to

CS 454 – Project Lab

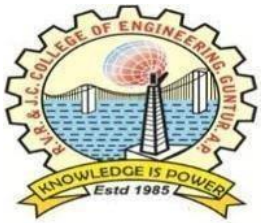
By

Batch No. 20

KOLLI HIMARAGA VARSHITH (Y23CS072)

KONGARA SRI VARSHITHA (Y23CS075)

PASUPULETI SRAVANI (Y23CS132)



R.V.R. & J.C. COLLEGE OF ENGINEERING (Autonomous)

(NAAC 'A+' Grade)

Approved by AICTE :: Affiliated to Acharya Nagarjuna University

Chandramoulipuram :: Chowdavaram

GUNTUR – 522019, Andhra Pradesh, India.

Sri. S Karthik	**Dr. Z Sunitha Bai**	**Prof N Nagamalleswara Rao**
Project Guide	Project In-Charge	Prof. & Head, CSE

Contents

1 Use Case Diagram **5**

1.1 Use Case Descriptions 5

2 Data Flow Diagram – Level 2 **7**

2.1 DFD Level 2 – Process 3.0 Code Generation 7

2.2 DFD Level 2 – Processes 4.0 and 5.0 Analysis, Review and Reflection 8

2.3 DFD Level 2 – Process 6.0 Evaluation and Reporting 9

2.4 Sub-process Details 9

3 Class Design **11**

3.1 Class Diagrams 11

3.2 Class Descriptions 14

3.2.1 BaseAgent (abstract) 14

3.2.2 ManagerAgent 15

3.2.3 ComplexityRouter 15

3.2.4 ArchitectAgent 15

3.2.5 DeveloperAgent 15

3.2.6 TesterAgent 16

3.2.7 ReviewerAgent 16

3.2.8 ReflectionAgent and AdaptiveStopper 16

3.2.9 Supporting Classes 16

4 Database Schema Design **18**

4.1 Entity Relationship Overview 18

4.2 Tables and Schemas 21

4.2.1 tasks 21

4.2.2 evaluations 21

4.2.3 runs 21

4.2.4 agent_events (append only) 21

4.2.5 artifacts 22

4.2.6 checkpoints . . . 22
4.2.7 model_calls . . . 22
4.2.8 metrics . . . 23
4.2.9 config_snapshots . . . 23
4.3 Indexes and Constraints . . . 23
4.4 Design Decisions . . . 23

5 API Design **25**
5.1 Endpoint Summary . . . 25
5.2 Detailed Specifications . . . 25
5.2.1 POST /runs . . . 25
5.2.2 GET /runs/{id} . . . 25
5.2.3 GET /runs/{id}/results . . . 25
5.2.4 POST /runs/{id}/change . . . 26
5.2.5 GET /runs/{id}/events . . . 26
5.2.6 WebSocket /ws/runs/{id} . . . 26
5.2.7 Error Format . . . 26
5.3 CLI Input and Output . . . 26

6 Sequence Diagrams **28**
6.1 Run on the SIMPLE Path . . . 28
6.2 STRUCTURED Path with Validation, Retry and Rollback . . . 29
6.3 Change Request Flow . . . 30
6.4 Benchmark and Statistical Comparison . . . 31
6.5 Life of a Run (State Diagram) . . . 32

7 Algorithms and Logic **33**
7.1 Complexity Routing Algorithm . . . 33
7.2 AST Dependency and Cycle Detection . . . 34
7.3 Context Slicing . . . 34
7.4 Adaptive Score and Stopping Rule . . . 35
7.5 Artifact Check, Retry and Rollback . . . 36

7.6 Change Request Impact 36
7.7 iCoder-Style Scoring (Neutral Evaluator) 37
7.8 Paired Permutation Test 37

8 Internal Communication **38**
8.1 Component-Level Communication 38
8.2 Communication Flows 38
8.3 Error Handling and Logging 38
8.4 Security Considerations 39

9 Code-Level Considerations **40**
9.1 Frameworks and Libraries 40
9.2 Project Structure 40
9.3 Naming and Conventions 40
9.4 Configuration 41
9.5 Extensibility 41
9.6 Reproducibility and Seeds 41
9.7 Testing Strategy 41
9.8 Hardware and Execution 42
9.9 Limitations 42

1. Use Case Diagram

MAS is made for one local user. The main actor is the Developer or Researcher. This person sends requirements, watches the runs and runs the experiments. The LLM provider and the ChatDev baseline are outside actors. The diagram below shows the use cases and how they are connected by include and extend.

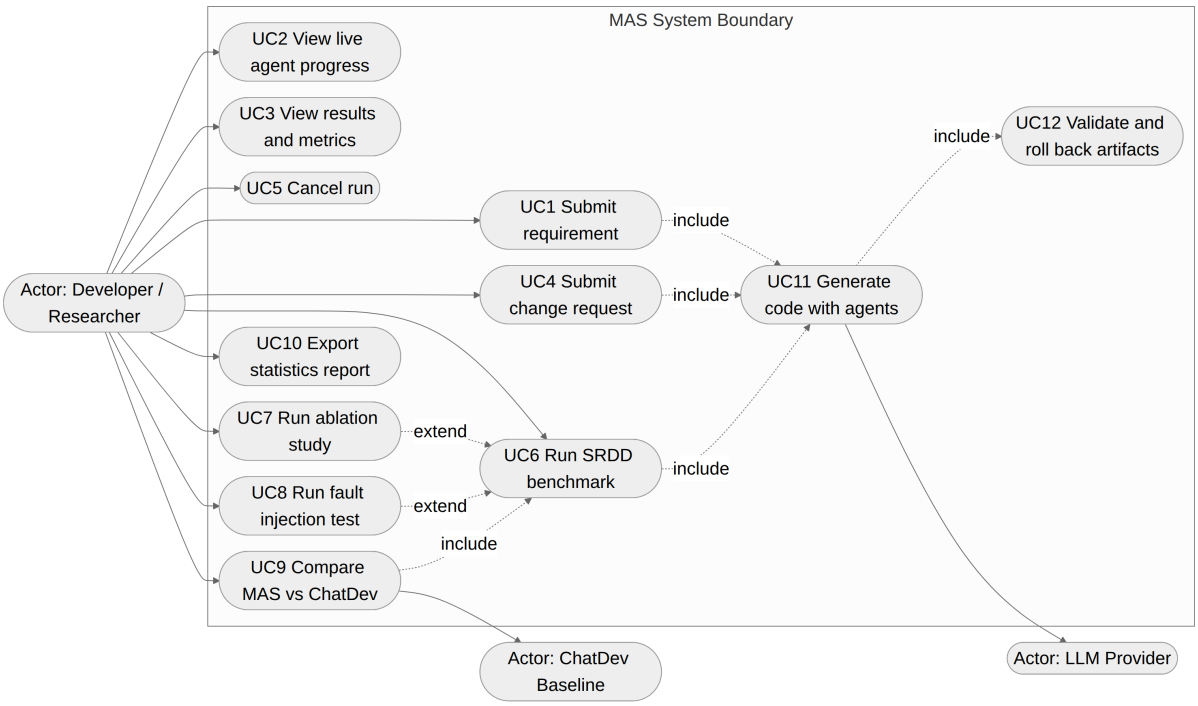


Figure 1: Use case diagram of MAS

1.1 Use Case Descriptions

ID	Use case	Actor	Main flow	Result
UC1	Submit requirement	Developer	Enter the requirement and model, the run is created and the workflow starts	Run saved with status RUNNING
UC2	View live agent progress	Developer	Open the CLI logs or the web page and receive WebSocket events	Events shown in order
UC3	View results and metrics	Developer	Ask for the results of a finished run	Metrics table and token chart shown
UC4	Submit change request	Developer	Enter the change, MAS finds the affected files, updates them and runs the regression tests	Updated code with regression and unused-code report

ID	Use case	Actor	Main flow	Result
UC5	Cancel run	Developer	Send cancel, the workflow stops at the next safe point	Status CANCELLED, partial data kept
UC6	Run SRDD benchmark	Researcher	Start the evaluation, MAS and ChatDev run on the tasks, then neutral scoring	Scores saved for all runs
UC7	Run ablation study	Researcher	Pick a feature flag, run 10 tasks twice and compare with full MAS	Ablation results saved
UC8	Run fault injection test	Researcher	Inject a broken UML or an incomplete story and watch detection and recovery	Detection and recovery metrics saved
UC9	Compare MAS and ChatDev	Researcher	Calculate the paired statistics	Comparison tables with p-values
UC10	Export statistics report	Researcher	Make CSV, JSON and plots	Report files created
UC11	Generate code with agents	(included)	Route, design, write, analyse, review and reflect	Final code and metrics saved
UC12	Validate and roll back artifacts	(included)	Validate, retry once, roll back, generate again	Only valid artifacts reach the next agent

2. Data Flow Diagram – Level 2

Level 2 opens up the level 1 processes that have the main logic of MAS. These are Code Generation (3.0), Analysis, Review and Reflection (4.0 and 5.0), and Evaluation (6.0).

2.1 DFD Level 2 – Process 3.0 Code Generation

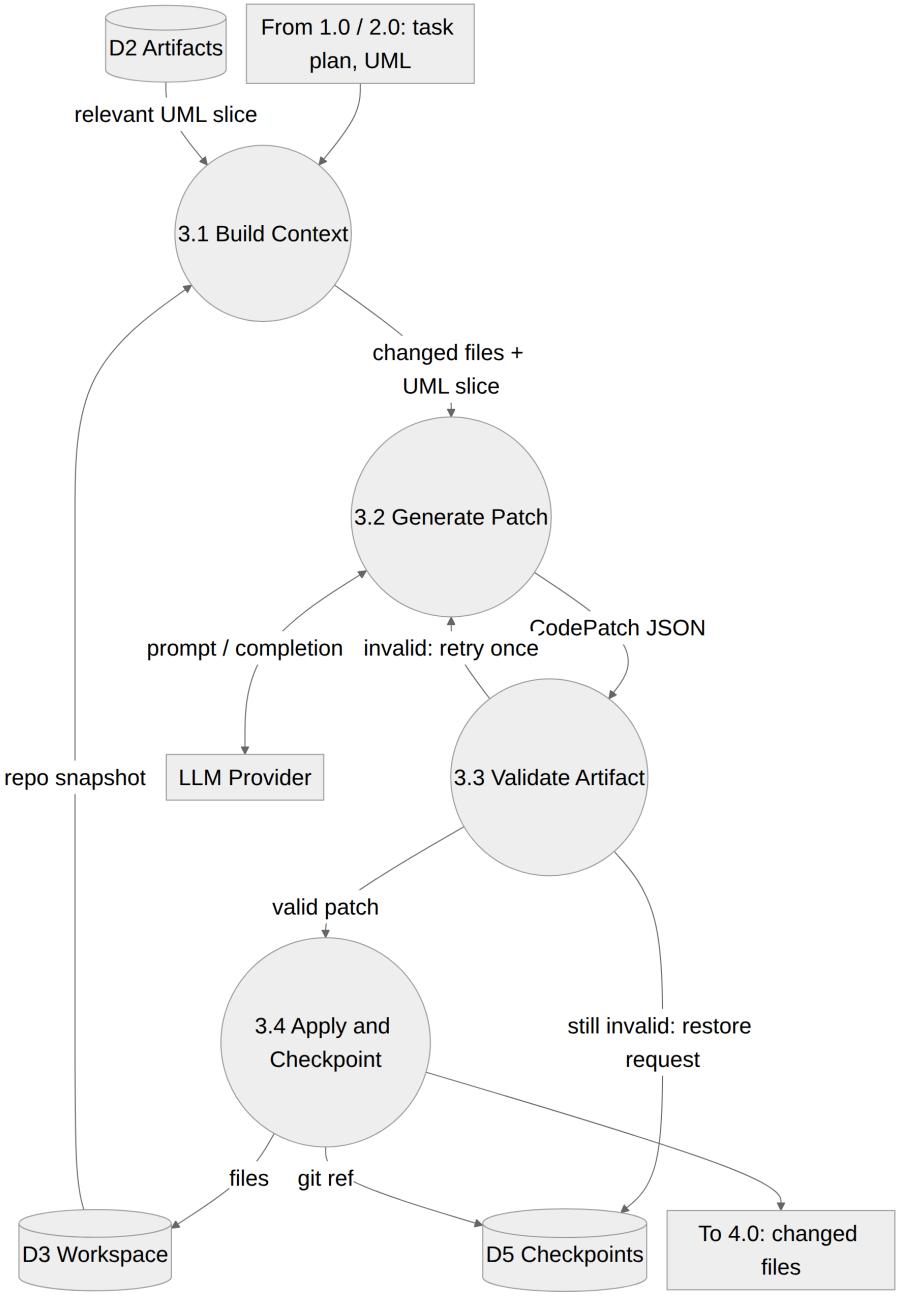


Figure 2: DFD Level 2 for process 3.0

2.2 DFD Level 2 – Processes 4.0 and 5.0 Analysis, Review and Reflection

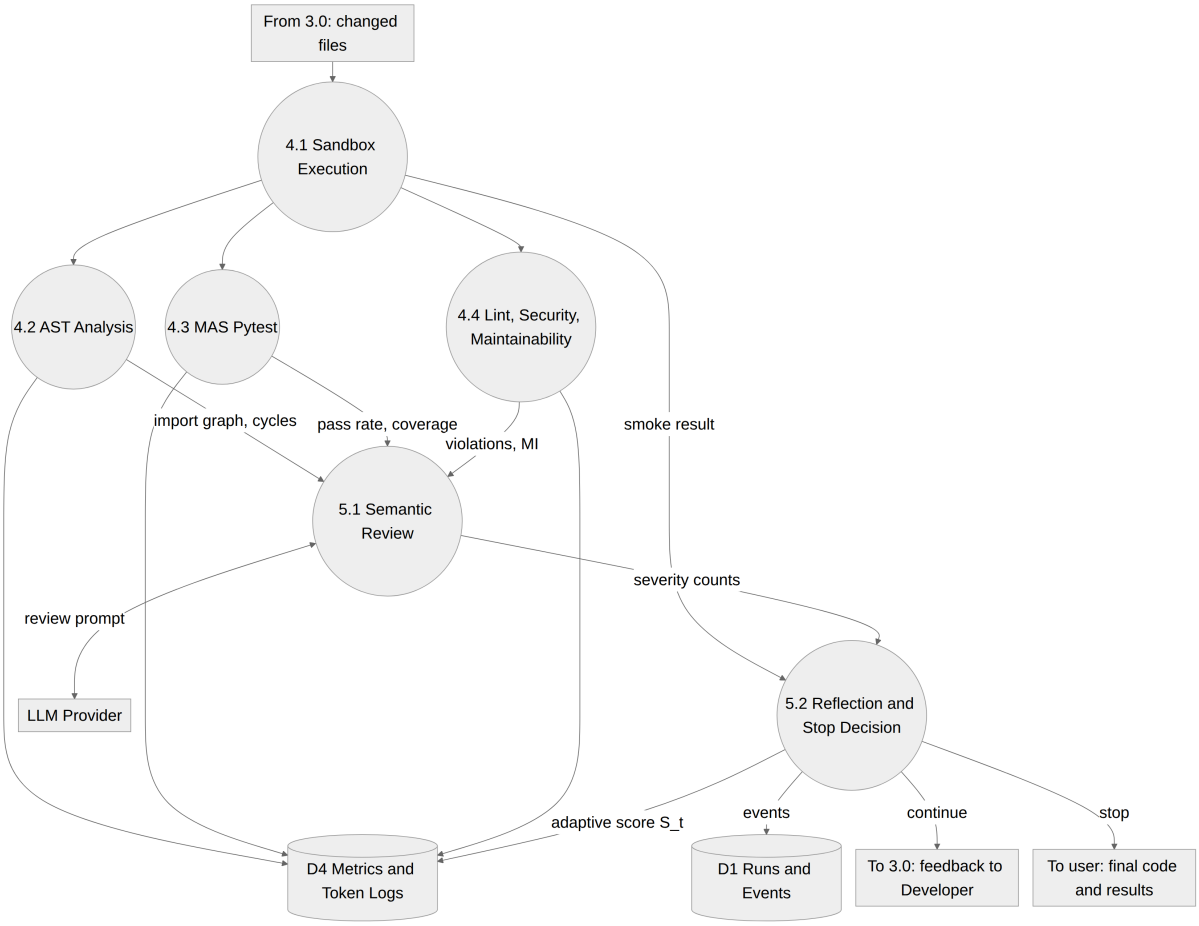


Figure 3: DFD Level 2 for processes 4.0 and 5.0

2.3 DFD Level 2 – Process 6.0 Evaluation and Reporting

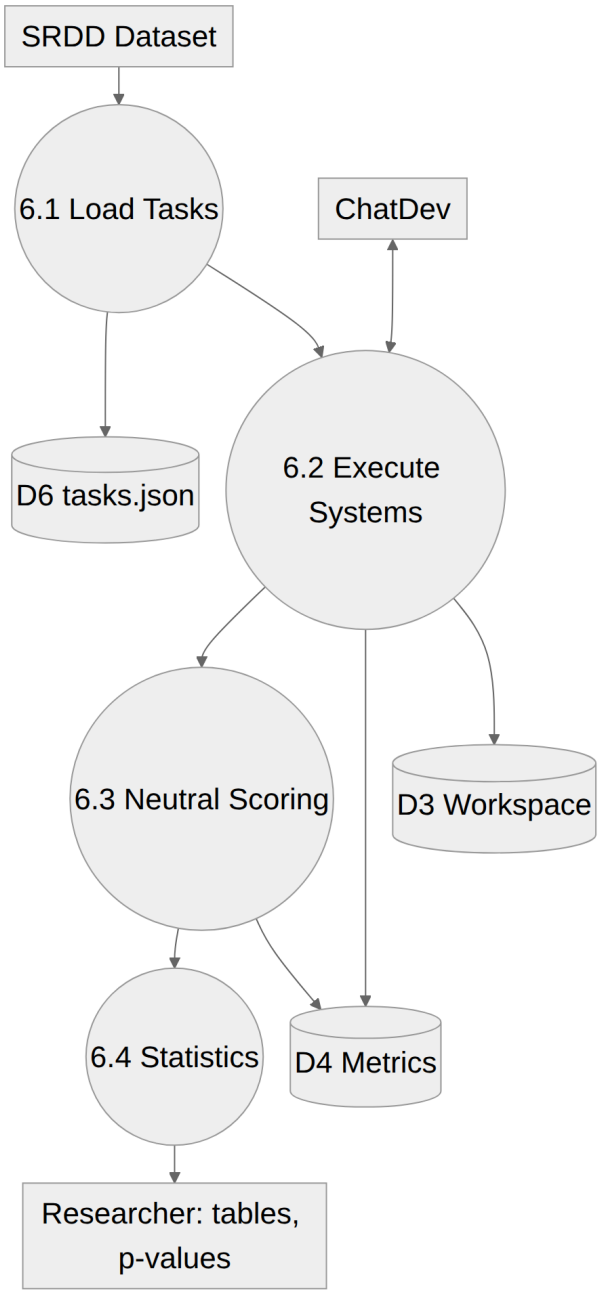


Figure 4: DFD Level 2 for process 6.0

2.4 Sub-process Details

Sub-process	Input	Output
3.1 Build Context	Repo snapshot, needed UML part, task plan	Changed files and UML part for the prompt
3.2 Generate Patch	Context and feedback	CodePatch JSON

Sub-process	Input	Output
3.3 Validate Artifact	CodePatch JSON	Valid patch, or a retry or rollback request
3.4 Apply and Checkpoint	Valid patch	Updated workspace and Git checkpoint
4.1 Sandbox Execution	Changed files	Smoke run result and logs
4.2 AST Analysis	Python files	Import graph, cycles, impact set
4.3 MAS Pytest	Workspace	Pass rate and coverage
4.4 Lint, Security, Maintainability	Python files	Ruff violations, Bandit findings, Radon index
5.1 Semantic Review	Patch and reports	Severity counts and comments
5.2 Reflection and Stop Decision	Reports and score history	Continue or stop with the reason
6.1 Load Tasks	SRDD dataset	tasks.json (20 tasks, fixed seed)
6.2 Execute Systems	tasks.json and model settings	Generated projects and token usage
6.3 Neutral Scoring	Projects	Quality, Shared Test, Ruff, Bandit, Radon
6.4 Statistics	Scores	Mean, SD, effect size, p-value, confidence interval

3. Class Design

In software engineering, class design is the backbone of an object-oriented system. This section lists the main classes of MAS. Each class has one clear job, like an agent, the router, the stopping rule, model access, the checks, the sandbox or the evaluation.

3.1 Class Diagrams

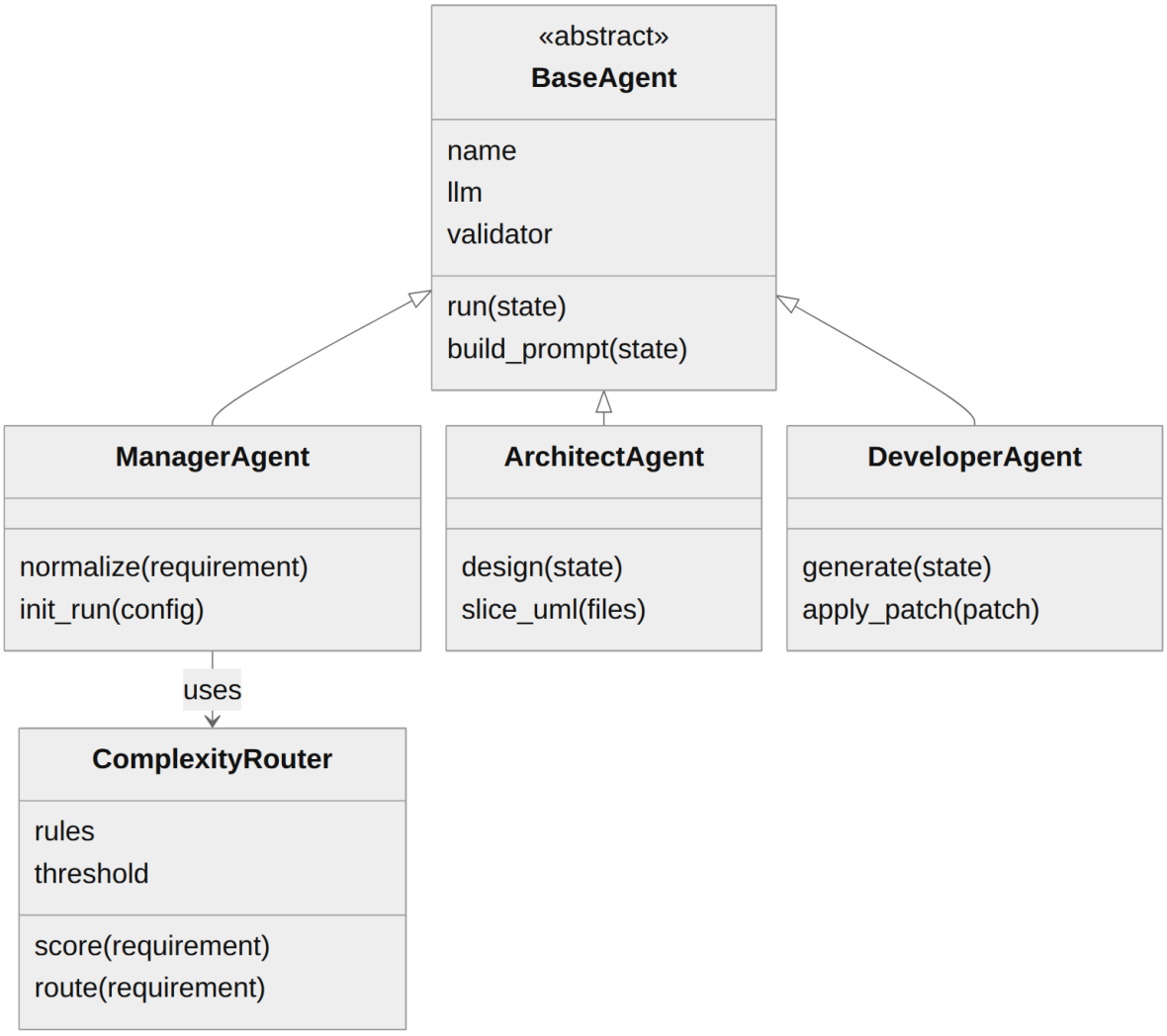


Figure 5: Class diagram: Manager, Router, Architect and Developer

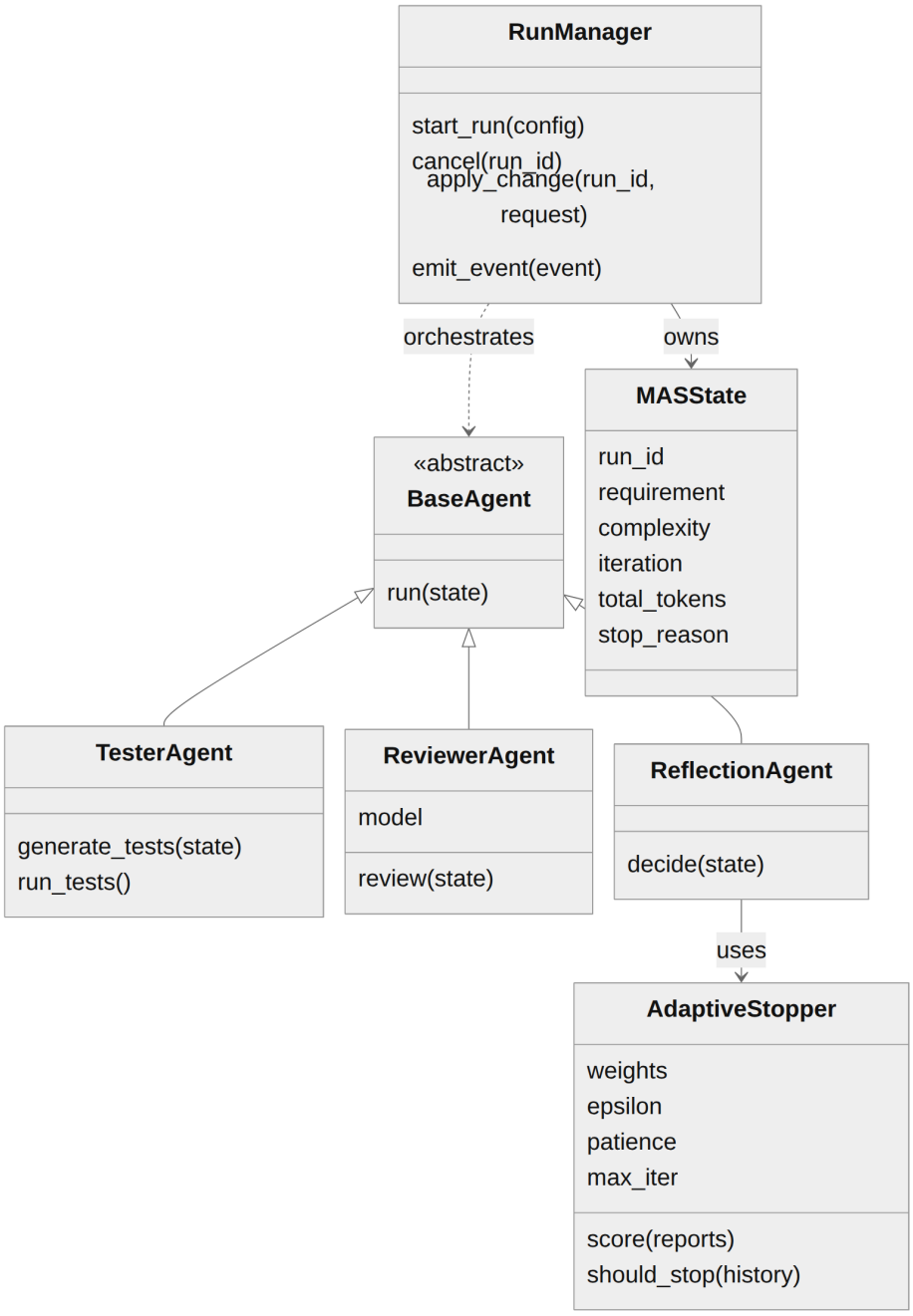


Figure 6: Class diagram: Tester, Reviewer, Reflection, stopper, run manager and state

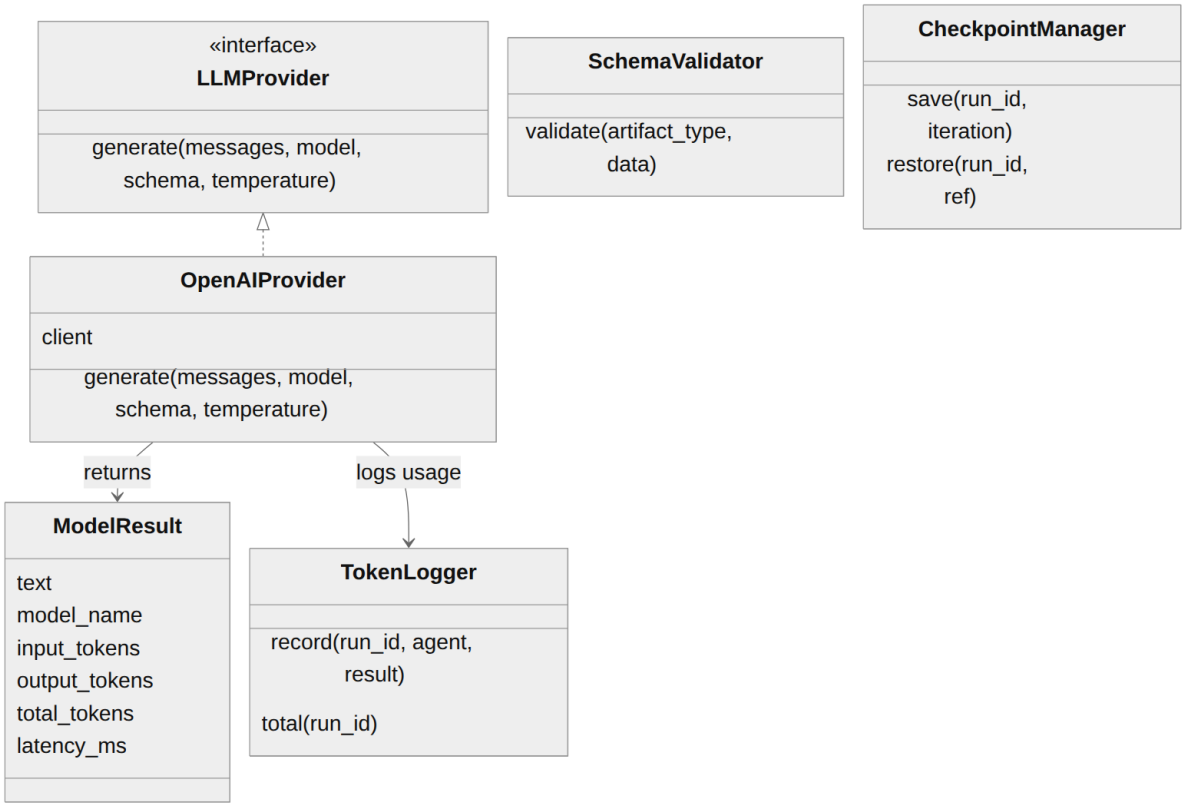


Figure 7: Class diagram: model access, validation and checkpoints

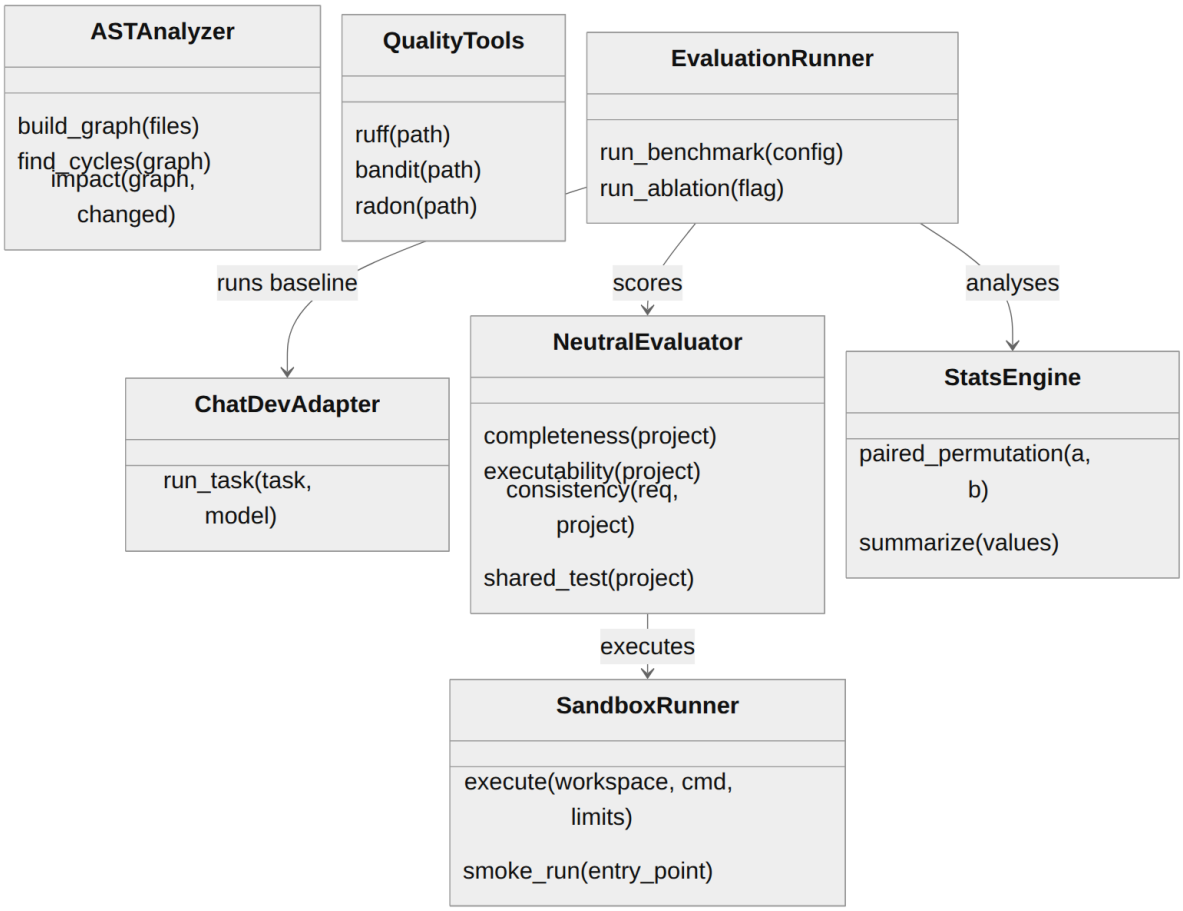


Figure 8: Class diagram: execution, analysis and evaluation

3.2 Class Descriptions

3.2.1 BaseAgent (abstract)

Purpose: The common base for all agents. It builds the prompt, calls the model and checks the output.
Attributes:

- `name`: name of the agent
- `llm`: the LLM provider object
- `validator`: the schema validator

Methods:

- `run(state)`: runs the agent and returns the updated state
- `build_prompt(state)`: builds a small prompt from the state

3.2.2 ManagerAgent

Purpose: Cleans up the requirement and creates the state and the run record.
Attributes: `config` – settings of the run (model, seed, flags).
Methods:

- `normalize(requirement)`: returns the TaskPlan
- `init_run(config)`: creates the MASState

3.2.3 ComplexityRouter

Purpose: Decides SIMPLE or STRUCTURED using fixed rules. No model call.
Attributes:

- `rules`: list of (feature, regex) pairs
- `threshold`: score from which a task is STRUCTURED (default 4)

Methods:

- `score(requirement)`: returns the score
- `route(requirement)`: returns the label, the score and the matched features

3.2.4 ArchitectAgent

Purpose: Makes the small UML design and sprint plan for STRUCTURED tasks and gives the needed UML part later.
Methods:

- `design(state)`: returns the Architecture artifact
- `slice_uml(files)`: returns the UML parts that matter for the given files

3.2.5 DeveloperAgent

Purpose: Writes the first code and the later patches from a small context.
Attributes: `workspace` – path of the project workspace.
Methods:

- `generate(state)`: returns a CodePatch
- `apply_patch(patch)`: writes the files and returns the list of changed files

3.2.6 TesterAgent

Purpose: Writes pytest tests and records the pass rate and the coverage for MAS's own use.
Methods:

- `generate_tests(state)`: creates the tests
- `run_tests()`: runs them in the sandbox and returns the TestReport

3.2.7 ReviewerAgent

Purpose: Checks the meaning of the code and the requirement match using GPT-4o-mini.
Methods: `review(state)` returns a ReviewReport with severity counts.

3.2.8 ReflectionAgent and AdaptiveStopper

Purpose: Calculates the score of each round and decides to continue or stop.
Attributes: `weights`, `epsilon` (0.02), `patience` (2), `max_iter` (3).
Methods:

- `score(reports)`: returns S_t between 0 and 1
- `should_stop(history)`: returns STOP or CONTINUE with the reason

3.2.9 Supporting Classes

Class	Purpose	Main methods
RunManager	Owns the runs and the state and sends events	start_run, cancel, apply_change, emit_event
MASState	The shared typed state of the workflow	(data class)
LLMProvider and OpenAIProvider	Model interface that calls the provider through the logging proxy	generate(messages, model, schema, temperature)
ModelResult	Text, usage, time and cost of one call	(data class)
TokenLogger	Saves every call and adds up the tokens of a run	record, total
SchemaValidator	Checks artifacts against JSON Schema	validate
CheckpointManager	Git checkpoints and rollback	save, restore
SandboxRunner	Runs code in Docker with limits	execute, smoke_run
ASTAnalyzer	Import graph, cycles and impact set	build_graph, find_cycles, impact

Class	Purpose	Main methods
QualityTools	Wrappers for Ruff, Bandit and Radon	ruff, bandit, radon
EvaluationRunner	Runs benchmarks and ablations	run_benchmark, run_ablation
ChatDevAdapter	Runs ChatDev on the same tasks and model	run_task
NeutralEvaluator	Hidden scoring, same for both systems	completeness, executability, consistency, shared_test
StatsEngine	Statistics for the comparison	paired_permutation, summarize

4. Database Schema Design

MAS uses PostgreSQL to store runs, events, artifacts, checkpoints, model calls and metrics. The generated source code is kept in the workspace and in Git. The database stores only references to it. The tables are normalised, so every number in the evaluation can be traced back to a run, a task, a system and its settings.

4.1 Entity Relationship Overview

The main entities are:

- `tasks`: the SRDD tasks
- `evaluations`: one benchmark or ablation batch
- `runs`: one execution of one system on one task
- `agent_events`: the event log, only new rows are added
- `artifacts`: outputs of the agents
- `checkpoints`: points to roll back to
- `model_calls`: token and cost log
- `metrics`: scores and diagnostics
- `config_snapshots`: record of the settings, for reproducibility

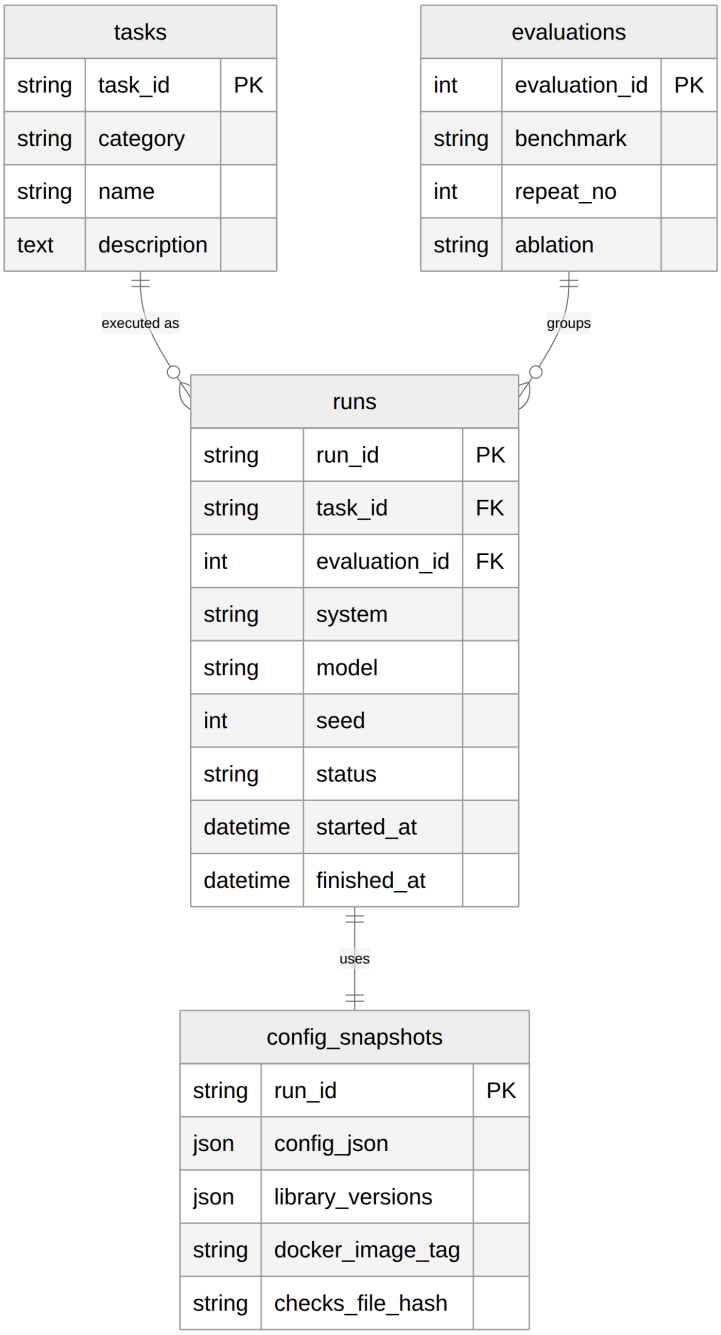


Figure 9: ER diagram (1/3): tasks, evaluations, runs and configuration

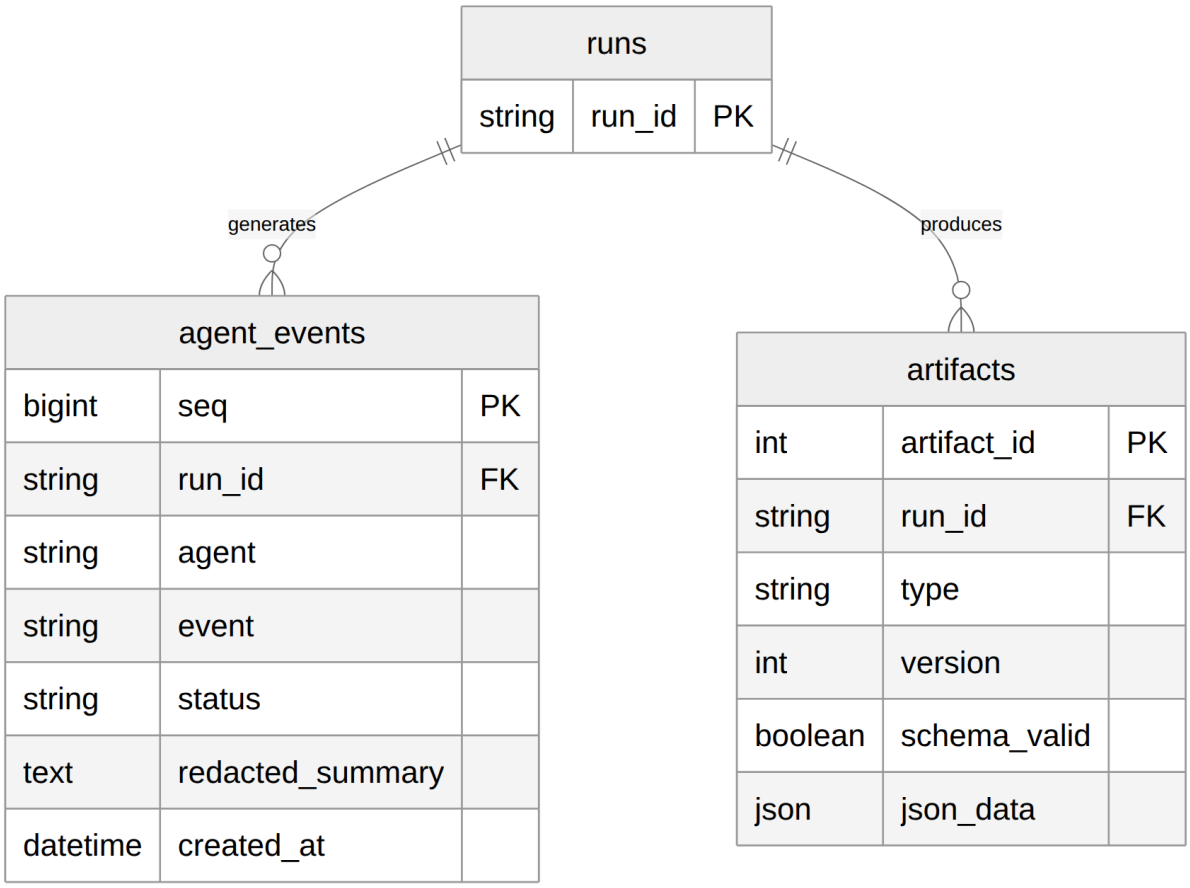


Figure 10: ER diagram (2/3): events and artifacts

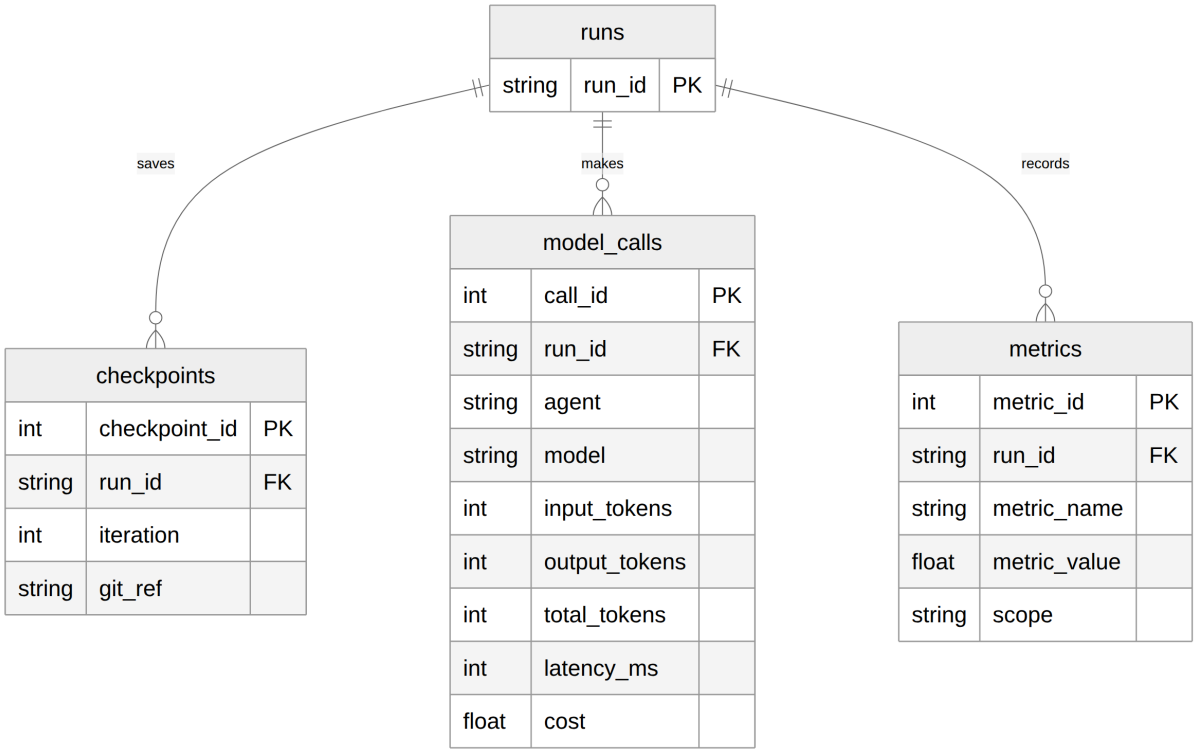


Figure 11: ER diagram (3/3): checkpoints, model calls and metrics

4.2 Tables and Schemas

4.2.1 tasks

Field	Data type	Description
task_id	VARCHAR(50) (PK)	Unique id of the task
category	VARCHAR(30)	Education, Work, Life, Games or Creation
name	VARCHAR(100)	Name of the software
description	TEXT	The requirement text

4.2.2 evaluations

Field	Data type	Description
evaluation_id	INT (PK)	Unique id of the batch
benchmark	VARCHAR(50)	For example srdd20
repeat_no	INT	Repeat number (1 to 3)
ablation	VARCHAR(30)	A0 (full MAS) or A1 to A6

4.2.3 runs

Field	Data type	Description
run_id	VARCHAR(40) (PK)	Unique id of the run
task_id	VARCHAR(50) (FK)	Task that was run (empty for a normal requirement)
evaluation_id	INT (FK)	Batch of the run (can be empty)
system	VARCHAR(20)	mas or chatdev
model	VARCHAR(40)	gpt-4o or gpt-4o-mini
seed	INT	Random seed
status	VARCHAR(20)	CREATED, RUNNING, COMPLETED, FAILED or CANCELLED
started_at, finished_at	TIMESTAMP	Start and end time

4.2.4 agent_events (append only)

Field	Data type	Description
seq	BIGINT (PK)	Number that always increases
run_id	VARCHAR(40) (FK)	The run
agent	VARCHAR(30)	Agent or tool name

Field	Data type	Description
event	VARCHAR(40)	run_started, agent_started, agent_completed, artifact_rejected, rollback, code_executed, requirement_changed, evaluation_started, evaluation_completed
status	VARCHAR(20)	success or failure
redacted_summary	TEXT	Short summary without secrets
created_at	TIMESTAMP	Time of the event

4.2.5 artifacts

Field	Data type	Description
artifact_id	INT (PK)	Unique id
run_id	VARCHAR(40) (FK)	The run
type	VARCHAR(30)	TaskPlan, UserStory, Architecture, CodePatch, ReviewReport, TestReport, ReflectionDecision
version	INT	Version of the artifact
schema_valid	BOOLEAN	Result of the schema check
json_data	JSON	The content

4.2.6 checkpoints

Field	Data type	Description
checkpoint_id	INT (PK)	Unique id
run_id	VARCHAR(40) (FK)	The run
iteration	INT	Round number
git_ref	VARCHAR(64)	Git commit hash of the workspace

4.2.7 model_calls

Field	Data type	Description
call_id	INT (PK)	Unique id
run_id	VARCHAR(40) (FK)	The run
agent	VARCHAR(30)	Agent that made the call
model	VARCHAR(40)	Model name
input_tokens, output_tokens, total_tokens	INT	Usage reported by the provider
latency_ms	INT	Time taken by the call
cost	FLOAT	Estimated cost from versioned price data

4.2.8 metrics

Field	Data type	Description
metric_id	INT (PK)	Unique id
run_id	VARCHAR(40) (FK)	The run
metric_name	VARCHAR(50)	quality, completeness, executability, consistency, shared_test, ruff, bandit, radon, pytest_pass_rate, coverage, adaptive_score
metric_value	FLOAT	The value
scope	VARCHAR(20)	neutral (used for comparing systems) or internal (MAS only)

4.2.9 config_snapshots

Field	Data type	Description
run_id	VARCHAR(40) (PK, FK)	The run
config_json	JSON	All settings used
library_versions	JSON	Python and package versions
docker_image_tag	VARCHAR(80)	Image used for running the code
checks_file_hash	VARCHAR(64)	Hash of the neutral evaluator checks file

4.3 Indexes and Constraints

- Every child table has a foreign key to `runs(run_id)` with ON DELETE CASCADE.
- Index on `agent_events(run_id, seq)` to read events in order.
- Index on `metrics(run_id, metric_name)` and on `runs(system, model, evaluation_id)` for quick results.
- The application role has no UPDATE or DELETE permission on `agent_events`, so it only grows.

4.4 Design Decisions

- Scores are saved for every run and never overwritten. So the statistics can be calculated again at any time.
- Neutral metrics (used to compare with ChatDev) and internal metrics (MAS only) are kept apart using the `scope` column. So MAS pytest results never enter the comparison.
- Token counts come only from the usage fields of the provider, saved in `model_calls`.
- MAS is for a single user, so there is no users table and no audit table. Login is future work.

- The configuration snapshot makes every result repeatable.

5. API Design

MAS gives a local REST API (FastAPI) with JSON bodies and a WebSocket for live events. There is no login in the local single-user MVP.

5.1 Endpoint Summary

Method	Endpoint	Description
POST	/runs	Create and start a run
GET	/runs/{id}	Status, complexity, round and token totals
GET	/runs/{id}/results	Metrics of a finished run
GET	/runs/{id}/events	Event log in order
POST	/runs/{id}/change	Send a change request
POST	/runs/{id}/cancel	Cancel a running run
GET	/evaluations/{id}	Results of a benchmark or ablation batch
WS	/ws/runs/{id}	Live event stream

5.2 Detailed Specifications

5.2.1 POST /runs

```
Request:
{
  "requirement": "Build a budget optimizer that records income and expenses ...",
  "model": "gpt-4o",
  "config_overrides": { "adaptive.max_iterations": 3 }
}
Response 201:
{ "run_id": "8f31c2d0", "status": "CREATED" }
```

5.2.2 GET /runs/{id}

```
Response 200:
{
  "run_id": "8f31c2d0", "status": "RUNNING",
  "complexity": "SIMPLE", "complexity_score": 2,
  "iteration": 2, "total_tokens": 18342, "estimated_cost": 0.071
}
```

5.2.3 GET /runs/{id}/results

```
Response 200:
{
  "run_id": "8f31c2d0", "stop_reason": "marginal_gain",
```

```
  "neutral":  { "quality": 0.0, "completeness": 0.0, "executability": 0,
                "consistency": 0.0, "shared_test": 0.0,
                "ruff": 0, "bandit": 0, "radon": 0.0 },
  "internal": { "pytest_pass_rate": 0.0, "coverage": 0.0, "adaptive_score": 0.0 },
  "tokens":   { "input": 0, "output": 0, "total": 0 },
  "quality_per_1k_tokens": 0.0
}
(the values above only show the format, they are not results)
```

5.2.4 POST /runs/{id}/change

```
Request:
{ "request": "Add task priorities and remove the old status field" }
Response 202:
{ "change_id": "c1", "status": "ANALYZING" }
```

5.2.5 GET /runs/{id}/events

```
Response 200:
[ { "seq": 1, "event": "run_started", "agent": "manager", "status": "success",
    "summary": "run created", "created_at": "2026-10-04T10:00:00Z" }, ... ]
```

5.2.6 WebSocket /ws/runs/{id}

```
Message:
{ "type": "agent.completed", "agent": "ast", "status": "success",
  "iteration": 1, "timestamp": "2026-10-04T10:00:07Z" }
Types: run.started | agent.started | agent.completed | artifact.invalid |
       rollback.started | test.completed | metric.updated | run.completed | run.
    failed
```

5.2.7 Error Format

```
{ "error": { "code": "RUN_NOT_FOUND", "message": "No run with id 8f31c2d0" } }
Status codes: 400 bad input, 404 not found, 409 wrong state (for example a change
              on a running run), 422 schema error, 500 internal error
```

5.3 CLI Input and Output

```
mas run --task "Build a task management API" --model gpt-4o
mas status RUN_ID        mas logs RUN_ID        mas results RUN_ID
mas change RUN_ID --request "Add task priorities"
mas evaluate --benchmark srdd20 --system mas --model gpt-4o --repeats 3
mas evaluate --benchmark srdd20 --system chatdev --model gpt-4o --repeats 3
mas ablate --feature router
```

```
Output (example format):
[01] Manager ok   [02] Router ok score=2 SIMPLE   [03] Developer ok 4 files
[04] AST ok cycles=0   [05] MAS pytest ok   [06] Ruff/Bandit/Radon ok
[07] Reviewer ok   [08] Reflection STOP: marginal gain below threshold
```

6. Sequence Diagrams

The sequence diagrams show how the users, the agents, the tools and the database talk to each other in the main flows.

6.1 Run on the SIMPLE Path

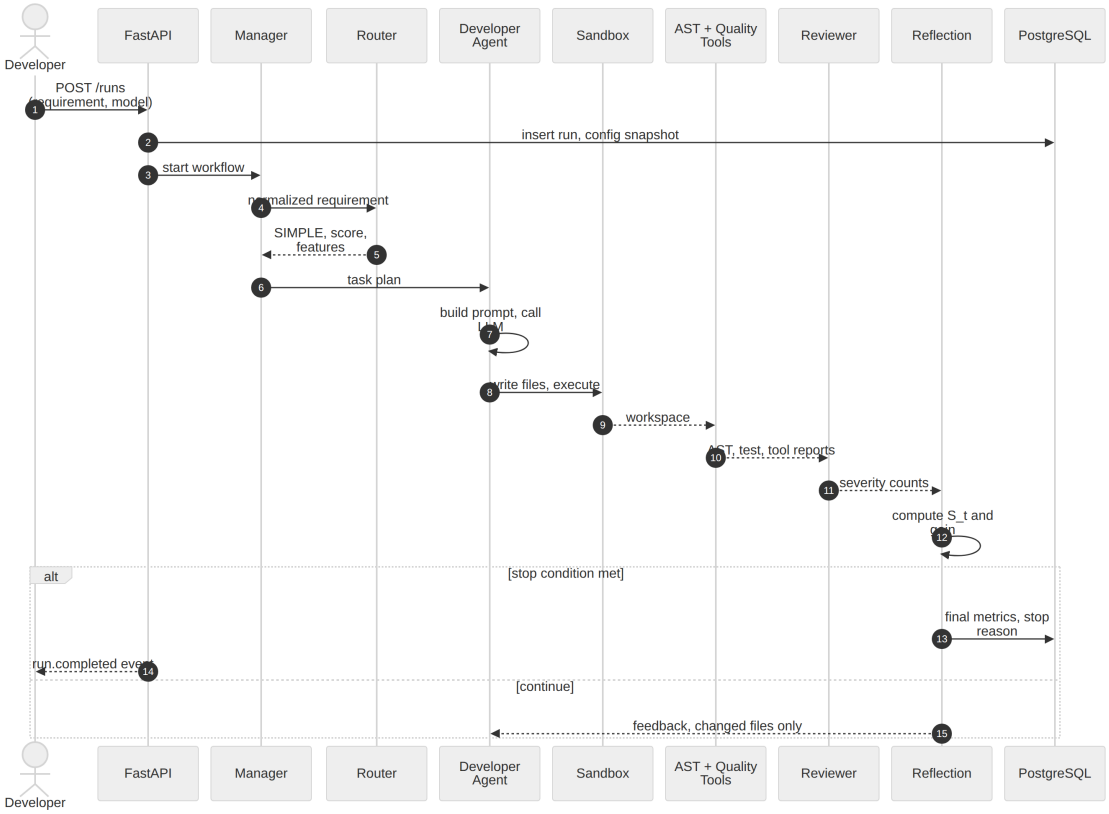


Figure 12: Sequence diagram: a run on the SIMPLE path

6.2 STRUCTURED Path with Validation, Retry and Rollback

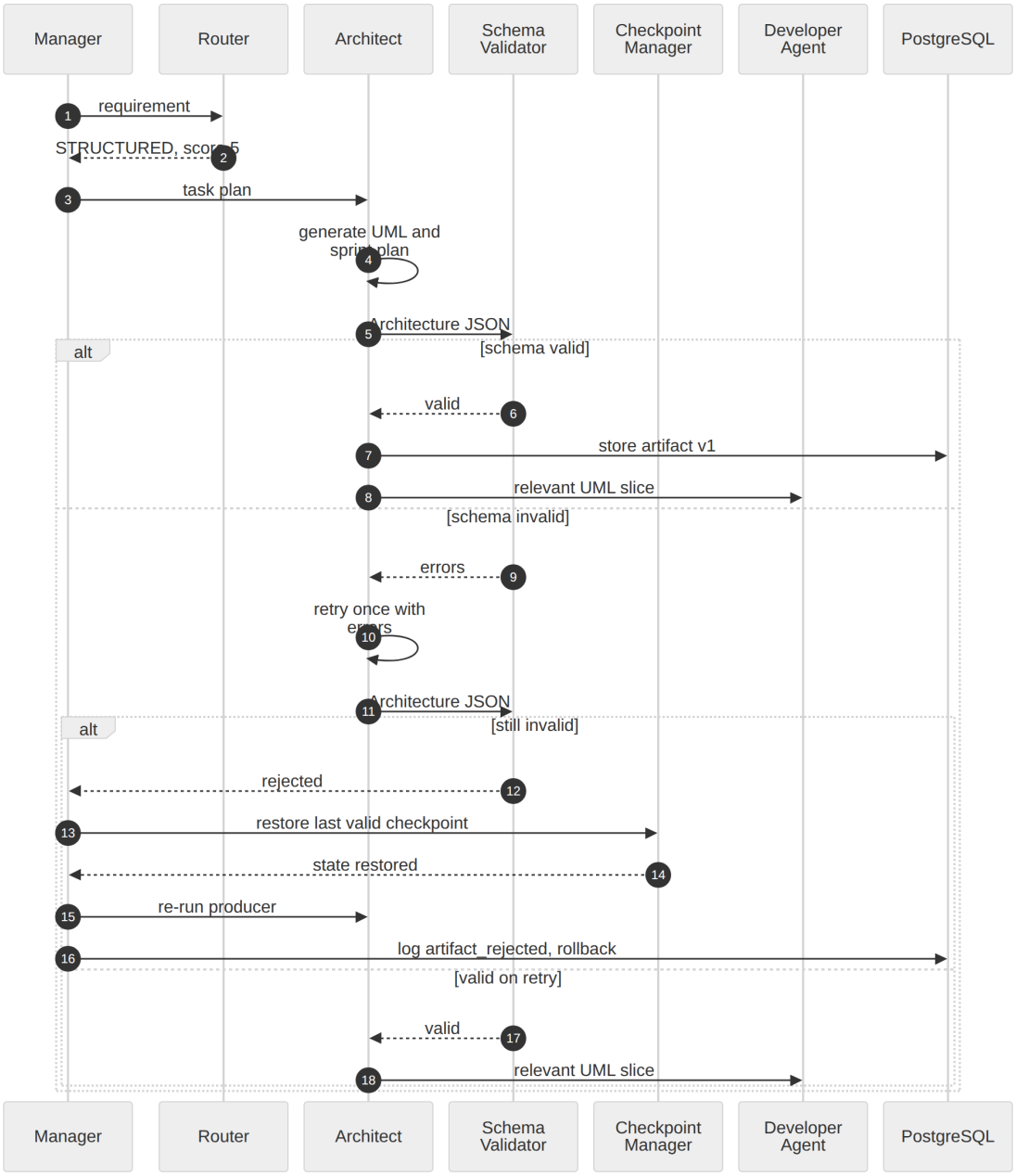


Figure 13: Sequence diagram: design step with artifact check and rollback

6.3 Change Request Flow

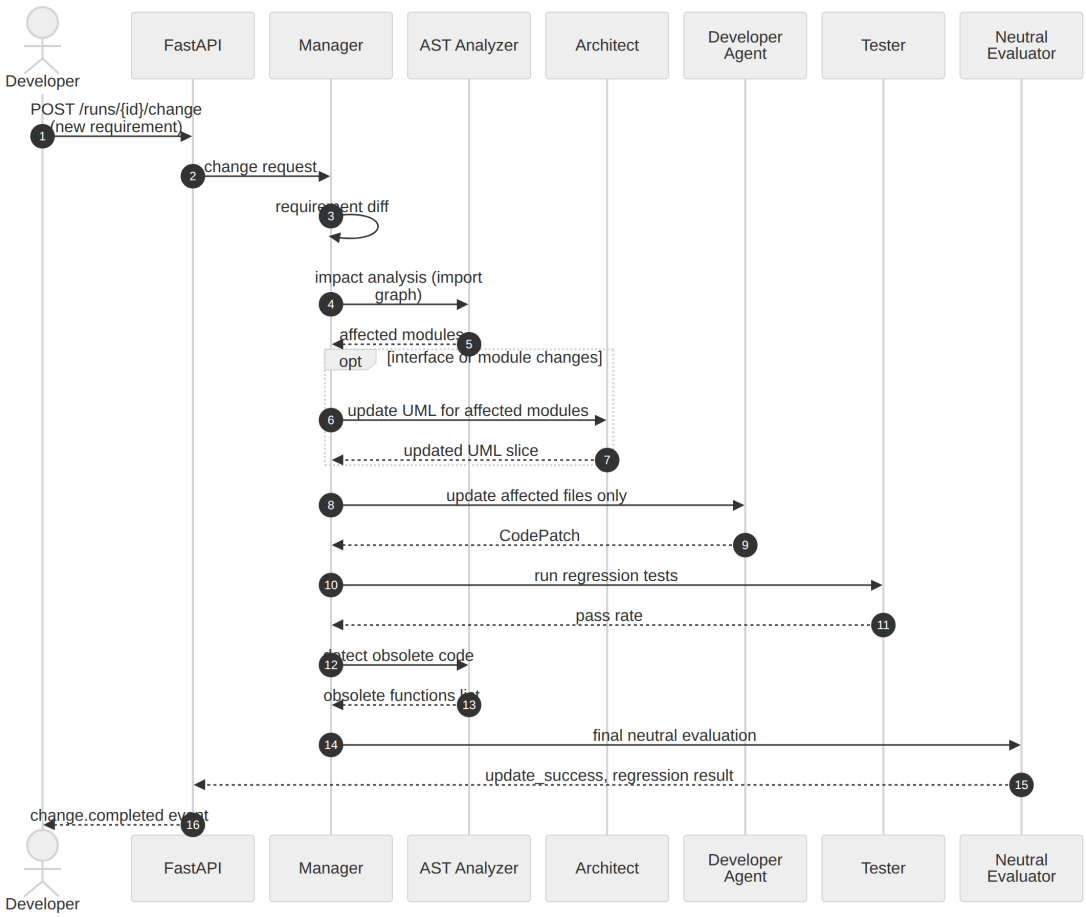


Figure 14: Sequence diagram: handling a change request

6.4 Benchmark and Statistical Comparison

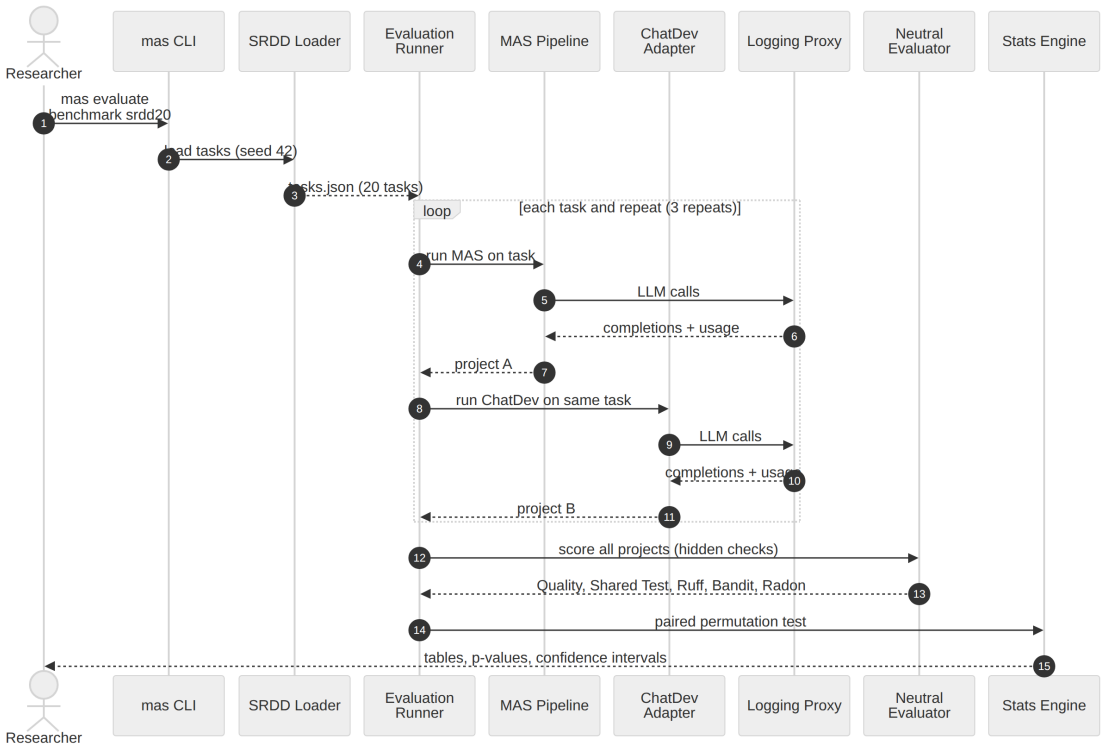


Figure 15: Sequence diagram: benchmark run, neutral scoring and statistics

6.5 Life of a Run (State Diagram)

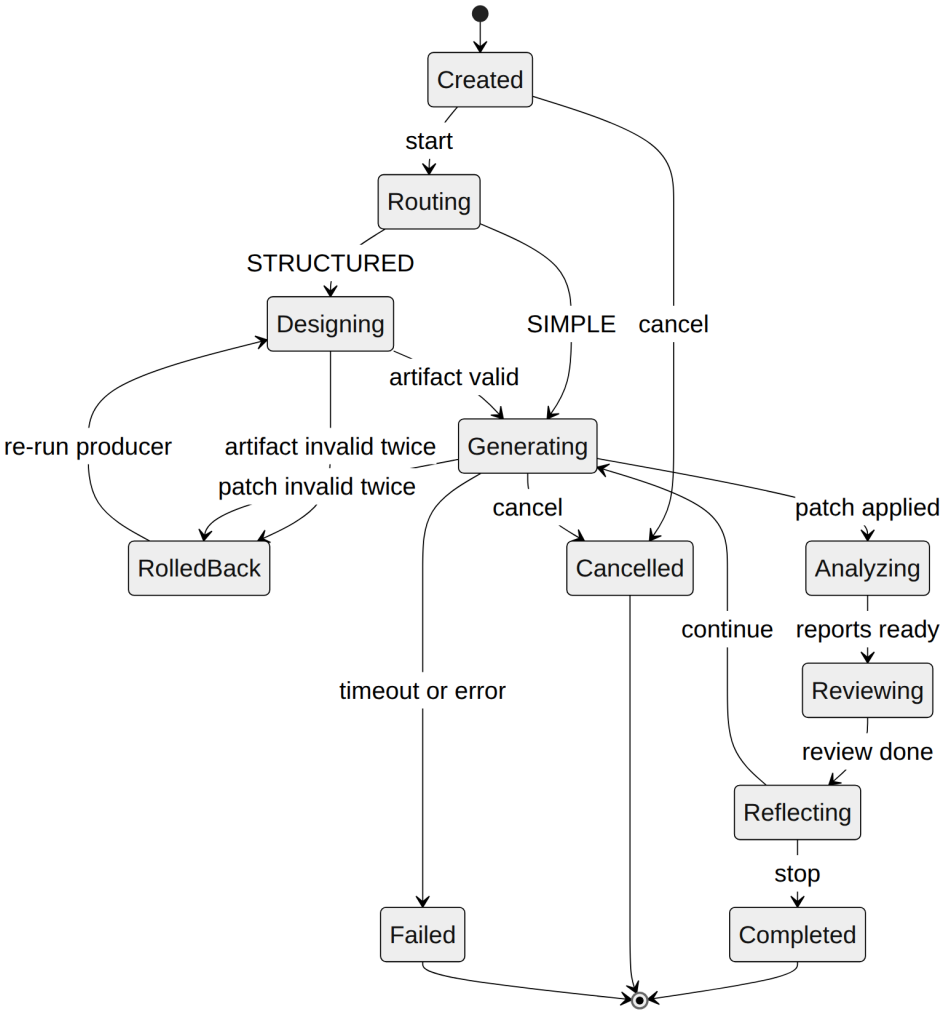


Figure 16: State diagram of a run

7. Algorithms and Logic

The intelligence of MAS is in a few simple algorithms. They decide when to spend tokens and when to use cheap checks.

7.1 Complexity Routing Algorithm

Purpose: To label a requirement as SIMPLE or STRUCTURED without calling a model.

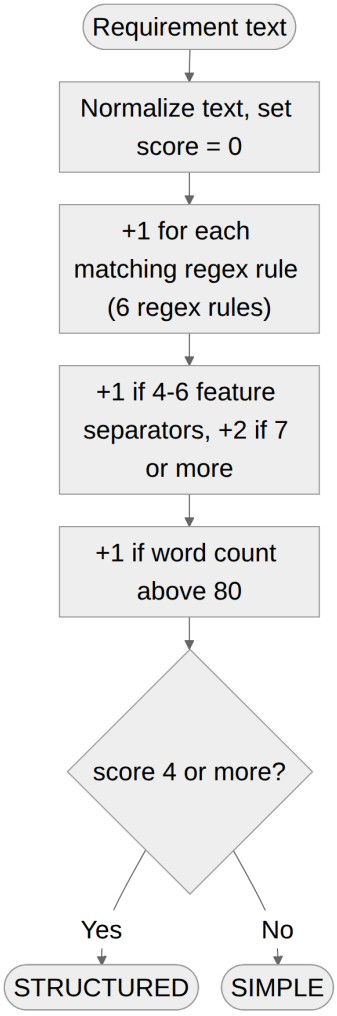


Figure 17: Flowchart of the routing algorithm

```
function ROUTE(requirement):
    text  <- lowercase(trim(requirement))
    score <- 0
    for each rule in RULES:                  # persistence, accounts, roles,
        if regex_search(rule.pattern, text):  # integration, analytics, workflow
            score <- score + 1
    n <- count_separators(text)              # commas, " and ", ";", bullets
    if n >= 7:      score <- score + 2
```

```
    else if n >= 4: score <- score + 1
    if word_count(text) > 80: score <- score + 1
    label <- "STRUCTURED" if score >= 4 else "SIMPLE"
    return (label, score, matched_features)
```

7.2 AST Dependency and Cycle Detection

Purpose: To find circular imports and missing imports, and to find which modules are affected by a change. No tokens are used.

```
function BUILD_GRAPH(files):
    graph <- {}
    for f in files:
        tree <- ast.parse(read(f))
        graph[module(f)] <- { resolve(imp) for imp in imports(tree) if internal(imp)
    }
        record unresolved imports of f
    return graph

function FIND_CYCLES(graph):
    color <- {v: WHITE for v in graph};  stack <- [];  cycles <- []
    procedure DFS(u):
        color[u] <- GRAY;  push(stack, u)
        for v in graph[u]:
            if color[v] == WHITE:  DFS(v)
            else if color[v] == GRAY:
                cycles.append(stack[index(v):] + [v])
        pop(stack);  color[u] <- BLACK
    for v in graph: if color[v] == WHITE: DFS(v)
    return cycles

function IMPACT(graph, changed):      # modules that import the changed modules
    reverse <- reverse_edges(graph)
    return transitive_closure(reverse, changed)
```

7.3 Context Slicing

Purpose: To send the Developer only the information it needs in each round.

```
function BUILD_CONTEXT(state):
    if state.iteration == 1:
        files <- all files needed by the task plan
    else:
        files <- state.changed_files UNION IMPACT(graph, state.changed_files)
    uml   <- slice_uml(state.architecture, files)      # STRUCTURED only
    return { task: state.requirement, files: contents(files), uml: uml,
             feedback: state.review_report.findings }
```

7.4 Adaptive Score and Stopping Rule

Purpose: To stop the loop when another round is not worth its tokens. Only the signals inside MAS are used. The neutral evaluator is never seen by the loop.

```
Each value below is between 0 and 1:
  InternalTest = MAS pytest pass fraction (0 if no tests ran)
  Ruff     = 1 - min(violations / 10, 1)
  Bandit   = 1 - min((3*High + 2*Medium + 1*Low) / 10, 1)
  Radon    = clip(mean maintainability index / 100, 0, 1)
  Reviewer = 1 - min((3*critical + 1*major) / 6, 1)
  CE       = 0.5 * (Completeness + Executability)     # internal smoke run

S_t = 0.30*InternalTest + 0.10*Ruff + 0.15*Bandit
    + 0.10*Radon + 0.15*Reviewer + 0.20*CE

function SHOULD_STOP(t, S, low_count, reports):
    delta <- S[t] - S[t-1]
    if no_critical(reports) and tests_pass(reports) and smoke_ok(reports):
        return STOP("clean_result")
    low_count <- low_count + 1 if delta < EPSILON else 0      # EPSILON = 0.02
    if low_count >= PATIENCE:   return STOP("marginal_gain")  # PATIENCE = 2
    if t >= MAX_ITERATIONS:     return STOP("max_iterations") # MAX = 3
    return CONTINUE
```

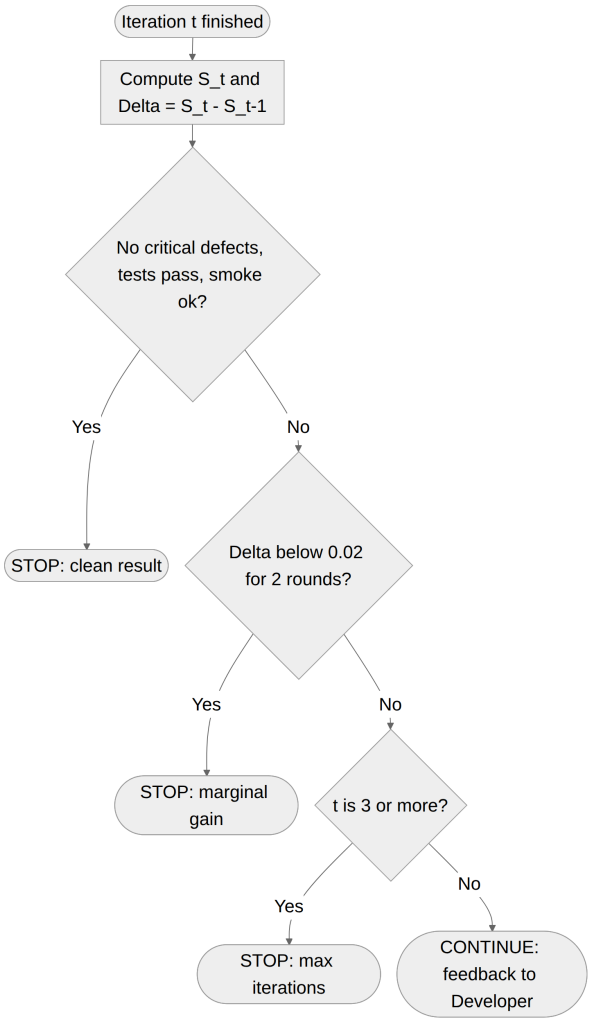


Figure 18: Flowchart of the stopping rule

7.5 Artifact Check, Retry and Rollback

```
function PRODUCE_VALID(producer, schema, state):
    ref <- CHECKPOINT.save(state)
    out <- producer(state)
    errors <- VALIDATE(schema, out)
    if errors is empty: return out
    out <- producer(state, errors)           # retry once, showing the errors
    if VALIDATE(schema, out) is empty: return out
    CHECKPOINT.restore(ref);  log("rollback")
    out <- producer(state)                   # try again from the last good state
    if VALIDATE(schema, out) is empty: return out
    fail_run("artifact_invalid")
```

7.6 Change Request Impact

```
function APPLY_CHANGE(run, new_requirement):
    diff     <- requirement_diff(run.requirement, new_requirement)
```

```
    terms    <- key_terms(diff.added, diff.removed)
    seeds    <- modules whose names, classes, functions or docstrings match terms
    affected <- seeds UNION IMPACT(graph, seeds)
    if interfaces_or_modules_change(diff): architecture <- ARCHITECT.update(affected)
    patch    <- DEVELOPER.generate(changed_files = affected, uml = slice(architecture
    ))
    apply(patch);  run regression tests
    obsolete <- functions defined but not used after the change
    return { update_success, regression_pass, obsolete_removed, tokens }
```

7.7 iCoder-Style Scoring (Neutral Evaluator)

```
Completeness_i   = 1 - unimplemented_functions_i / total_functions_i   # pass or TODO
Executability_i  = 1 if the project starts under the neutral smoke test, else 0
Consistency_i    = (cos(emb(requirement), emb(code)) - min) / (max - min)
                   # min and max taken over all projects of both systems together
Quality_i        = Completeness_i * Executability_i * Consistency_i
Quality          = mean of Quality_i over all projects
SharedTest_i     = 0.5*Level1_i + 0.5*(Level-2 pass fraction)   # only Level1 if no
    Level-2 checks
Quality per 1k tokens = Quality / (mean total tokens / 1000)
```

7.8 Paired Permutation Test

```
function PERMUTATION_TEST(a, b, trials = 10000):   # a, b: task-level means
    d <- [a_i - b_i for each task i]
    observed <- abs(mean(d))
    count <- 0
    repeat trials times:
        flipped <- [s_i * d_i with s_i chosen randomly from {-1, +1}]
        if abs(mean(flipped)) >= observed: count <- count + 1
    return (count + 1) / (trials + 1)        # two-sided p-value
```

8. Internal Communication

The parts of MAS talk to each other through the typed state, checked artifacts and a small number of clear interfaces. This section gives a short summary.

8.1 Component-Level Communication

Component	Talks to	Protocol or interface
CLI and React page	FastAPI server	REST (HTTP) and WebSocket
FastAPI server	LangGraph workflow	Function calls inside the same process
Agents	Shared MASState	Typed Python objects (Pydantic)
Agents	LLM provider	Provider interface through the local logging proxy (HTTPS to the provider)
Workflow	Schema validator and checkpoint manager	Function calls, JSON Schema checks and Git commands
Workflow	Docker sandbox	Docker API, one container per run of the code
Workflow and harness	PostgreSQL	SQL through SQLAlchemy
ChatDev adapter	Logging proxy	OpenAI-style HTTP requests
Evaluation runner	MAS and ChatDev	CLI or API calls with the same tasks and settings

8.2 Communication Flows

1. **Run execution:** The API creates the run and the state. Then the flow is Manager, Router, Architect (if needed), Developer, sandbox and tools, Reviewer and Reflection. Every step creates an event that is saved and streamed.

2. **Artifact exchange:** Every agent gives a JSON artifact. The schema validator accepts or rejects it before the next agent can read it. Agents share short structured results and not their full reasoning.

3. **Token counting:** Every model call goes through the proxy. The usage fields are saved in `model_calls` and added up per run, for both systems.

4. **Evaluation:** The runner collects the finished projects. The neutral evaluator, which the agents cannot see, scores them and saves metrics with scope neutral. MAS-only checks are saved with scope internal.

8.3 Error Handling and Logging

- Logging is central and uses Python logging. Secrets are removed from event summaries.

- Wrong artifacts: retry once, roll back, generate again, and then fail the run with a clear reason.
- Model time-outs and broken outputs are retried with a wait time and then logged as run failures.
- A run can take at most 20 minutes. Failed and timed-out runs stay in the results.
- The event log only grows, so it can be used for checking and debugging.

8.4 Security Considerations

- MAS is a local single-user system. There is no login or role system in the MVP (future work).
- Generated code runs only in Docker with no network, 1 CPU, 512 MB memory, limited processes, a 30 to 60 second time limit, a read-only base filesystem and no host folders except the workspace.
- Prompt injection control: system rules come first, then task data, then the user requirement (kept inside markers and not trusted), then the agent instructions. Artifacts are treated as data and not as instructions.
- API keys come from environment variables or a file that Git ignores. We commit `.env.example` and not `.env`.
- The generated code never gets any secret.

9. Code-Level Considerations

This section lists the main points we follow while writing the code, so that it stays easy to maintain, repeatable and fast.

9.1 Frameworks and Libraries

- LangGraph for the agent workflow, FastAPI for the API, Pydantic for typed artifacts and SQLAlchemy for PostgreSQL.
- Python `ast`, Ruff, Bandit, Radon, pytest and coverage.py for the checks.
- sentence-transformers (all-MiniLM-L6-v2), NumPy, pandas and SciPy for the evaluation.
- Docker and Docker Compose for running and deployment.

9.2 Project Structure

```
mas/
  agents/    manager.py router.py architect.py developer.py tester.py reviewer.py
    reflection.py
  core/      state.py schemas.py validator.py checkpoints.py llm.py proxy.py events.
    py
  analysis/  ast_analyzer.py quality_tools.py sandbox.py
  api/       main.py routes.py websocket.py
  db/        models.py migrations/
  cli/       main.py
eval/
  tasks.json  srdd_loader.py  runner.py  chatdev_adapter.py
  scorers/   icoder_metrics.py shared_test.py mas_pytest_diagnostic.py
  checks/    level2_checks.json
  stats.py  reports.py
frontend/    React and Tailwind run page
config.yaml  docker-compose.yml  .env.example
```

9.3 Naming and Conventions

- snake_case for functions and variables, PascalCase for classes, and short clear file names.
- Pydantic models are the one source for artifacts. JSON Schema is made from them for the runtime check.
- Type hints and docstrings on all public functions.

9.4 Configuration

```
adaptive:
  weights: {internal_test: 0.30, ruff: 0.10, bandit: 0.15, radon: 0.10, reviewer:
    0.15, ce: 0.20}
  stop_epsilon: 0.02
  patience: 2
  max_iterations: 3
router: {simple_max: 3}
models: {main: gpt-4o, cheap: gpt-4o-mini, reviewer: gpt-4o-mini}
seed: 42
run_timeout_minutes: 20
```

9.5 Extensibility

- New models: write a class for the LLMProvider interface and add the model name in the settings.
- New tools: add a wrapper in the analysis folder and a rule to bring its value between 0 and 1 for the adaptive score.
- New programming languages: add an analyser for that language (future work).
- New ablations: add a feature flag and read it in the matching component.

9.6 Reproducibility and Seeds

- Fixed seed (42) for picking tasks and for the permutation tests. Temperature 0 for model calls.
- Every run saves the model name and version, library versions, Docker image tag, settings and the hash of the neutral evaluator checks file.
- Each task is run 3 times. We report mean $\pm$ SD and use a paired permutation test ($\alpha = 0.05$) on the main metrics, which are Quality and Quality per 1,000 tokens.

9.7 Testing Strategy

- Unit tests for the router, the AST analyser (with a made-up cycle), the schema validator, the adaptive stopper and the scorers.
- Integration test: one SIMPLE and one STRUCTURED task from start to end in the sandbox.
- Fault tests: a broken UML and an incomplete user story must cause a retry and a rollback.
- A pilot with one task is run before the full batch of 300 runs.

9.8 Hardware and Execution

- A normal laptop or desktop with Docker is enough. No GPU is needed, because the models are used through the API and the embedding model is small.
- Benchmark tasks run in parallel containers. The planning budget is about 10.8 million tokens for the 300 core runs. We will replace it with the numbers measured in the pilot.

9.9 Limitations

- Most SRDD tasks may be routed as SIMPLE, so the UML and sprint steps may run less often.
- The comparison with iCoder is indirect, because we use only its published results. The direct comparison is with ChatDev.
- Only 20 tasks are used, so the statistical power is low. The embedding-based consistency score is a weak way to measure correctness.
- The neutral evaluator uses only a smoke test for programs with a window, and hand-written checks for some of the CLI and API tasks.